

C 题示例：数据驱动预测、评价与优化

摘 要： 本文针对【研究对象与核心任务】，依次完成数据预处理、特征分析、模型构建与结果检验。针对问题一，采用【方法】得到【关键结果】；针对问题二和问题三，分别建立【模型】并给出【结论】。敏感性分析表明【稳定性结论】。本文方法可为【应用场景】提供决策依据。

关键词： 数据预处理；机器学习；预测评价；优化决策

一、 问题重述

1.1 问题背景

在此概括研究背景、业务场景及建模目标，并说明问题属于预测、评价、分类或优化中的哪一类。

1.2 问题提出

问题1： 在此说明问题一的输入、输出与评价目标；

问题2： 在此说明问题二需要建立的模型及约束；

问题3： 在此说明问题三的预测、优化或推广任务。

二、 问题分析

2.1 问题一分析

在此给出“数据检查—特征构造—模型训练—结果检验”的分析链条。

2.2 问题二分析

在此说明模型选择依据和问题之间的数据传递关系。

2.3 问题三分析

在此说明验证方案、敏感性分析和结论解释方式。

三、 模型假设

1. 假设【数据记录或抽样过程】能够代表研究对象的主要特征；
2. 假设【研究期内的关键关系】保持相对稳定；
3. 假设【未观测因素】可通过误差项或敏感性分析进行评估。

四、 符号说明

表 1 主要符号说明

符号	含义	单位
x_{ij}	第 i 个样本在第 j 个特征上的原始观测值	—
z_{ij}	数据预处理后的特征值	—
θ	待估计的模型参数向量	—
$\hat{y}_i$	第 i 个样本的预测值	题目给定单位
L	模型的损失函数或评价指标	—

五、 数据预处理

5.1 数据质量检查

在此汇总数据规模和质量，并给出可复现的检查规则。

5.2 缺失值处理

在此说明缺失值识别、处理方法及其对结果的影响。

5.3 异常值检测

在此说明异常值判据、人工复核过程和最终保留策略。

5.4 数据标准化

以效益型指标为例，可写为

$$z_{ij} = \frac{x_{ij} - \min_i x_{ij}}{\max_i x_{ij} - \min_i x_{ij}}. \quad (1)$$

5.5 特征工程与探索性分析

在此说明特征构造与筛选依据。根据题型，可进一步选择回归、分类、聚类、时间序列或机器学习模型。

六、 问题一模型建立与求解

6.1 问题分析

在此给出问题一的技术路线。图 1 可替换为实际算法流程图。

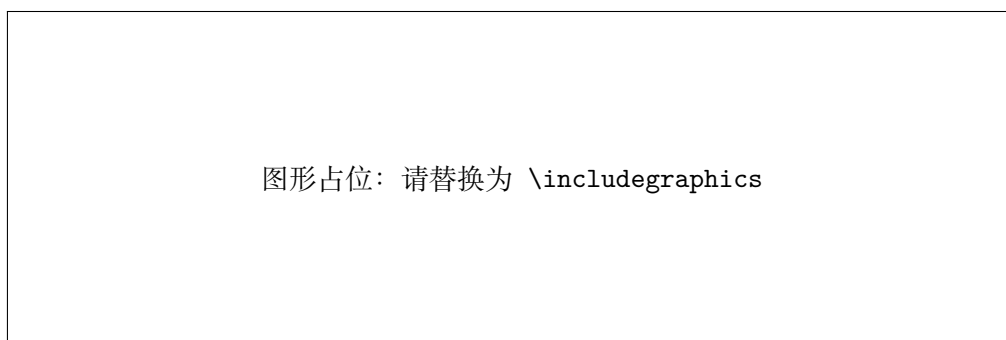


图 1 问题一建模流程图

6.2 数据处理

在此引用第五节的数据预处理结果，并说明问题一的专用处理步骤。

6.3 模型建立

用占位表达式表示预测或评价模型：

$$\hat{y}_i = f(\mathbf{x}_i; \boldsymbol{\theta}), \quad \boldsymbol{\theta}^* = \arg \min_{\boldsymbol{\theta}} L(\boldsymbol{\theta}). \quad (2)$$

6.4 模型求解

在此描述训练、搜索或优化过程，并把完整可运行程序放入附录与支撑材料。

6.5 结果分析

表 2 问题一结果占位表

模型	训练集指标	验证集指标	备注
基准模型	【填写】	【填写】	用于比较
改进模型	【填写】	【填写】	最终模型

七、 问题二模型建立与求解

7.1 问题分析

在此说明问题二的新增条件、目标和评价标准。

7.2 模型建立

在此给出模型结构、目标函数和约束条件，并解释每个参数的实际意义。

7.3 模型求解

在此记录完整求解流程，保证结果可以由附录程序复现。

7.4 结果分析

在此给出核心结果、图表引用和具有业务意义的解释。

八、 问题三模型建立与求解

8.1 问题分析

在此说明问题三的决策变量、约束和不确定性来源。

8.2 模型建立

在此写出目标函数、约束条件及模型假设。

8.3 模型求解

在此说明参数设定、求解流程及计算复杂度。

8.4 结果分析

在此给出问题三的定量结论和可执行建议。

8.5 模型检验与敏感性分析

在此报告误差指标、稳健性结果和参数变化的临界区间，并说明模型失效条件。

九、 模型评价

9.1 模型优点

在此概括模型相较基准方法的主要优势，并用前文结果支撑判断。

9.2 模型缺点

在此说明模型依赖的条件、可能的偏差及尚未解决的问题。

9.3 模型推广

在此给出模型推广场景、必要修改和后续研究方向。

AI 工具使用声明

本参赛队在竞赛过程中未使用任何 AI 工具。

参考文献

- [1] Hwang, C. L., and Yoon, K. *Multiple Attribute Decision Making: Methods and Applications*. Springer, 1981.

A 支撑材料文件列表

- code/problem1.py、code/problem2.py、code/problem3.py: 完整可运行源程序;
- code/problem1.m: Matlab 程序示例;
- data/external-data.xlsx: 自主查询并实际使用的数据 (如有);
- results/intermediate-results.xlsx: 支撑结论的必要中间结果 (如有);
- AI 工具使用详情.pdf: 仅在真实使用 AI 工具时按官方规定提供。

B 完整源程序代码

代码 1 问题一完整源程序

```
1  """Problem 1: compute min-max normalized weighted scores."""
2
3
4  def normalize_columns(rows):
5      columns = list(zip(*rows))
6      normalized = []
7      for row in rows:
8          current = []
9          for value, column in zip(row, columns):
10             low, high = min(column), max(column)
11             current.append(0.0 if high == low else (value - low) / (high - low))
12         normalized.append(current)
13     return normalized
14
15
16 def weighted_scores(rows, weights):
17     if any(len(row) != len(weights) for row in rows):
18         raise ValueError("Each row must have the same length as weights")
19     return [sum(value * weight for value, weight in zip(row, weights)) for row in rows]
20
21
22 if __name__ == "__main__":
23     data = [[8.0, 6.0, 7.0], [7.0, 9.0, 6.0], [6.0, 7.0, 9.0]]
24     result = weighted_scores(normalize_columns(data), [0.4, 0.35, 0.25])
25     print([round(value, 4) for value in result])
```

代码 2 问题二完整源程序

```
1 """Problem 2: solve a small integer resource-allocation example."""
2
3
4 def best_allocation(scores, capacities, budget):
5     if len(scores) != len(capacities):
6         raise ValueError("scores and capacities must have equal length")
7     best_value, best_plan = float("-inf"), None
8
9     def search(index, remaining, plan):
10         nonlocal best_value, best_plan
11         if index == len(scores):
12             value = sum(score * amount for score, amount in zip(scores, plan))
13             if value > best_value:
14                 best_value, best_plan = value, plan.copy()
15             return
16         for amount in range(min(capacities[index], remaining) + 1):
17             search(index + 1, remaining - amount, plan + [amount])
18
19     search(0, budget, [])
20     return best_plan, best_value
21
22
23 if __name__ == "__main__":
24     allocation, value = best_allocation([0.842, 0.766, 0.693], [3, 4, 5], 7)
25     print({"allocation": allocation, "objective": round(value, 4)})
```

代码 3 问题三完整源程序

```
1 """Problem 3: summarize score ranges under weight perturbations."""
2
3
4 def normalize_weights(weights):
5     total = sum(weights)
6     if total <= 0:
7         raise ValueError("The weight sum must be positive")
8     return [weight / total for weight in weights]
9
10
11 def score_ranges(normalized_data, base_weights, relative_change=0.1):
12     scenarios = []
13     for index in range(len(base_weights)):
14         for direction in (-1.0, 1.0):
15             changed = base_weights.copy()
16             changed[index] *= 1.0 + direction * relative_change
17             weights = normalize_weights(changed)
18             scenarios.append([
19                 sum(value * weight for value, weight in zip(row, weights))
20                 for row in normalized_data
21             ])
22     return [(min(values), max(values)) for values in zip(*scenarios)]
```

```

23
24
25 if __name__ == "__main__":
26     data = [[1.0, 0.2, 0.5], [0.5, 1.0, 0.0], [0.0, 0.4, 1.0]]
27     ranges = score_ranges(data, [0.4, 0.35, 0.25])
28     print([(round(low, 4), round(high, 4)) for low, high in ranges])

```

代码 4 问题一 Matlab 程序示例

```

1 % Problem 1 Matlab placeholder.
2 % Replace the sample data and method with the team's complete implementation.
3 data = [8, 6, 7; 7, 9, 6; 6, 7, 9];
4 weights = [0.40; 0.35; 0.25];
5 normalized = normalize(data, 'range');
6 scores = normalized * weights;
7 disp(scores);

```